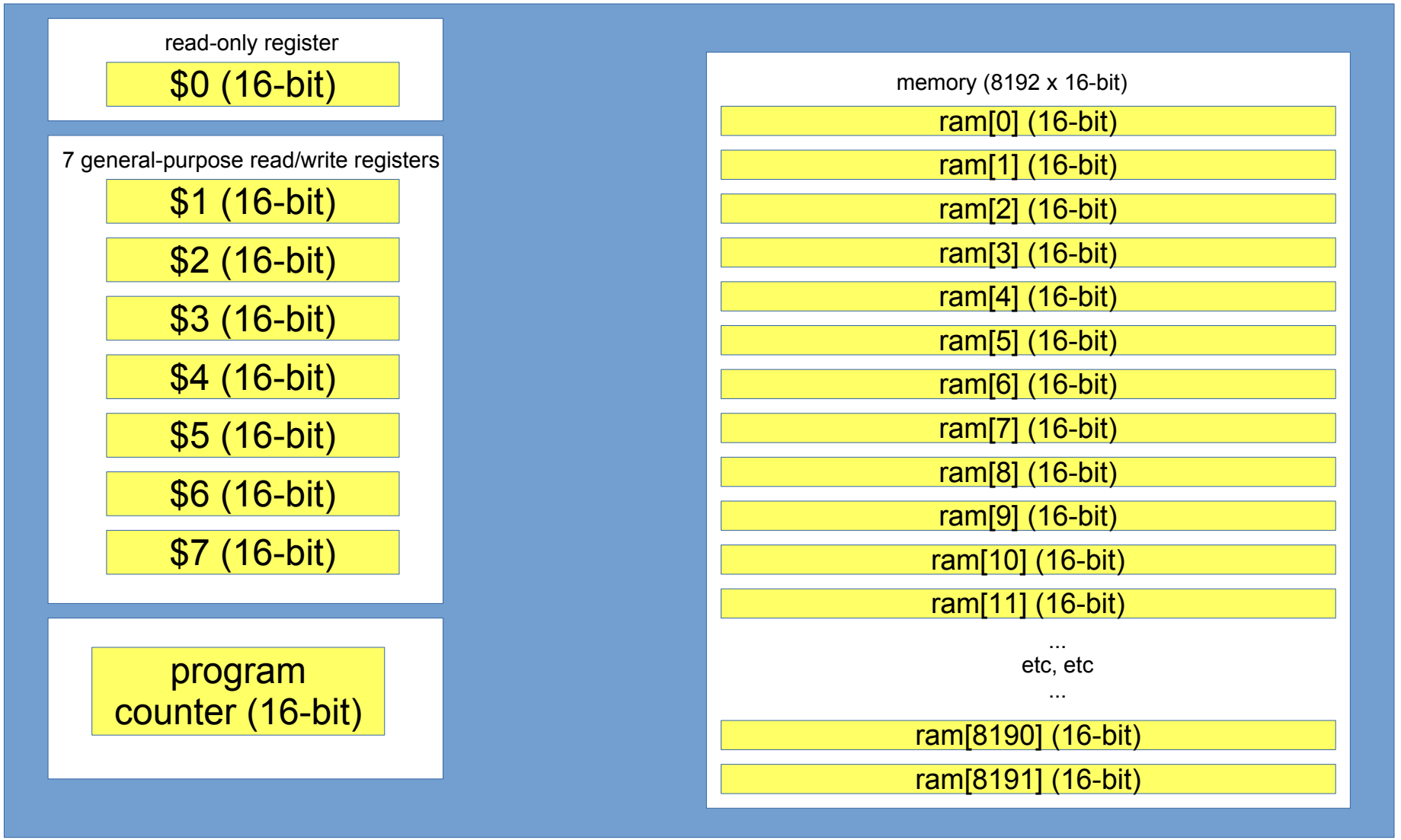
Today's Lecture Plan

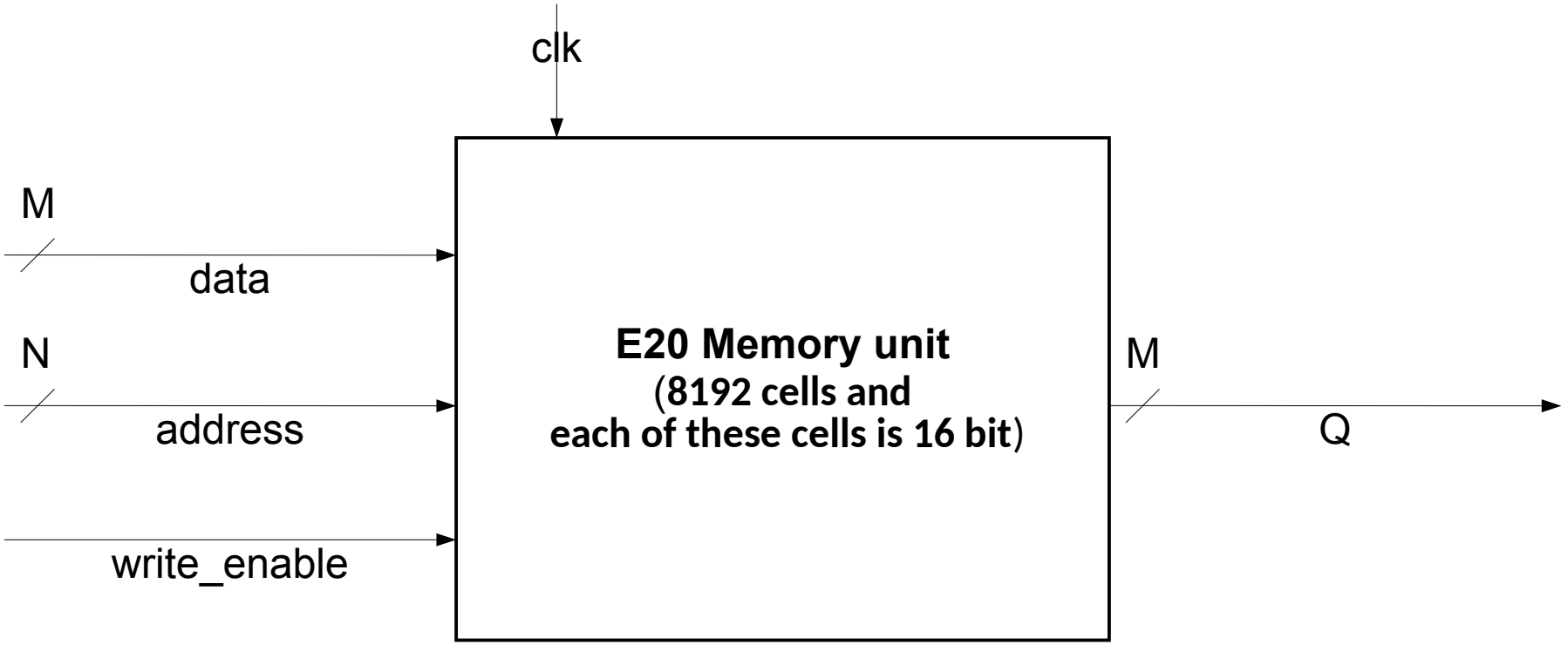
- Discussion of HW3 problems.
- Introduction to E20
    - Registers
    - Memory
    - Program Counter
- Comparison between E15 and E20.
- E20 Assembly language Instructions.
- E20 Machine Code Translation.
-

E20

- Architecture is more powerful than E15.
- E20 Assembly language is more expressive.
- Encoding of machine language is more complicated.

What's inside the E20 processor?

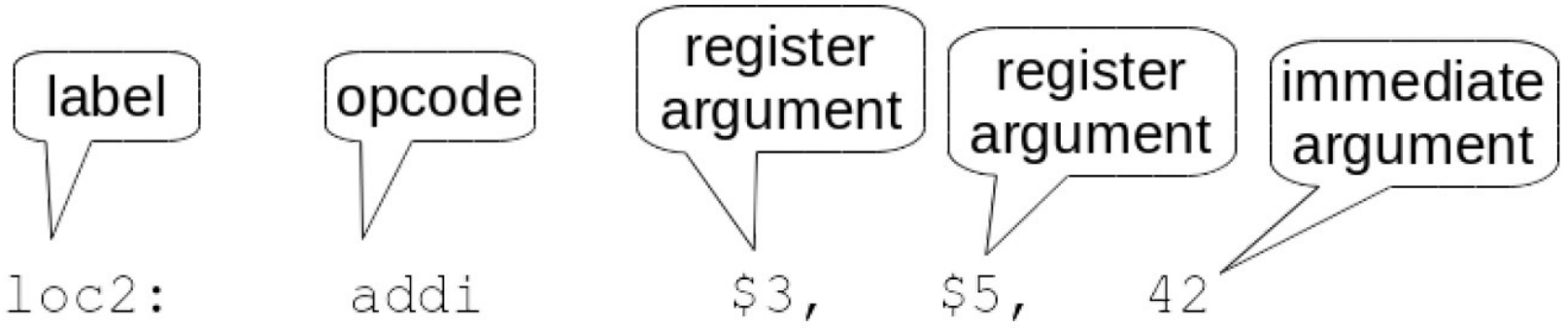



clk
M
data
N
address
write_enable
E20 Memory unit
(8192 cells and
each of these cells is 16 bit)
M
Q

Comparison between E15 and E20 processors?

	E15	E20
registers	four 4-bit registers	seven 16-bit registers
memory	16 12-bit cells of read-only instruction memory	8192 16-bit cells of instruction/-data memory
instruction format	all instructions have 4-bit opcode field, two 2-bit register fields, and 4-bit immediate field	three formats: • 3-bit opcode field, three 3-bit register fields, 4-bit immediate field • 3-bit opcode field, two 3-bit register fields, 7-bit immediate field • 3-bit opcode field, 13-bit immediate field

E20 Assembly Instruction Format



E20 Assembly Instructions

- Instructions with three register arguments
  - add $regDst, $regSrcA, $regSrcB
  - sub $regDst, $regSrcA, $regSrcB
  - and $regDst, $regSrcA, $regSrcB
  - or $regDst, $regSrcA, $regSrcB
  - slt $regDst, $regSrcA, $regSrcB
  - jr $reg (remaining two operands will be filled with zero register) – Jump Instructions
- Instructions with two register arguments
  - slti $regDst, $regSrc, imm
  - addi $regDst, $regSrc, imm
  - jeq $regA, $regB, imm – Jump Instructions
  - lw $regDst, imm($regAddr) – Memory Reference Instruction
  - sw $regSrc, imm($regAddr) – Memory Reference Instruction
- Instructions with no register arguments
  - j imm – Jump Instructions
  - jal imm – Jump Instructions
-

E20 Assembly Instructions (Pseudo Instructions)

- movi $reg, imm
  - The movi $reg, imm instruction is translated by the assembler as

    addi $reg, $0, imm.
- nop
  - The nop instruction is translated by the assembler as

    add $0, $0, $0.
- Halt
  - The halt instruction is translated by the assembler as an unconditional jump (j) to the current memory location.

    endofprogram: j endofprogram
  -

E20 Assembler Directives

- .fill imm

  Inserts a 16-bit immediate value directly into memory at the current location.

  This directive instructs the assembler to put a number into the place where an instruction would normally be.

  The immediate value may be specified numerically (positive or negative or zero) or with a label.

  -

  -

-

label

- A label is a symbol that represents a known memory address.
- A label name is a sequence of characters
  - The first character may be a letter or an underscore
  - Subsequent characters may be a letter, an underscore, or a digit
  - There must be at least one character.
  - Label names are not case-sensitive.
- A label may be declared in E20 assembly code by giving its name followed by a colon, preceding any instruction (including normal instructions, pseudo-instruction, and directives).
- The value of the label will be the address of the instruction that it precedes.
  - If the label precedes no instruction, the value of the label will be the address that a subsequent instruction would occupy
- A label's name may be used as the imm field of any instruction, assuming that the value of the label can be expressed in the number of bits allocated to that field.
- Labels make it easier to write E20 assembly language programs. Why? – Please go to pollev.com/ratandeyto write your answer.

Example of using label in E20 Assembly Code

```
first_label:
    movi $1, 1
    j first_label
    j second_label
second_label:
    movi $2, 2
```

Example of using label in E20 Assembly Code

```
first_label:
    movi $1, 1
    j first_label
    j second_label
second_label:
    movi $2, 2
```

```
ram[0] = 16'b0010000010000001;
ram[1] = 16'b0100000000000000;
ram[2] = 16'b0100000000000011;

ram[3] = 16'b0010000100000010;
```

Here's a sample of E20 assembly language.
Answer the questions based on your intuition.
How is this similar to E15 code?
How is this different from E15 code?

```
# Some simple math stuff

addi $1, $0, 5        # $1 := 5
addi $2, $1, -2       # $2 := $1 + (-2)
add $3, $1, $2        # $3 := $1 + $2

addi $4,$0, 55        # $4 := 55
sub $5, $4,$1         # $5 := $4 - $1
sub $4, $5, $4        # $4 := $5 - $4

or $6, $2, $5
and $7,$2, $5

halt                  # end the program
```

What does this program do?
What is the final value of each register?

Here's a sample of E20 assembly language.

How is this similar to E15 code?
How is this different from E15 code?

```
# A simple loop, counting down from 10

    movi $1,10          # Initialize counter to 10
beginning:
    jeq $1, $0,done      # if $1 == $0, go to done
    addi $1, $1,-1       # Decrement $1
    j beginning          # go to top of loop
done: halt               # we've finished
```

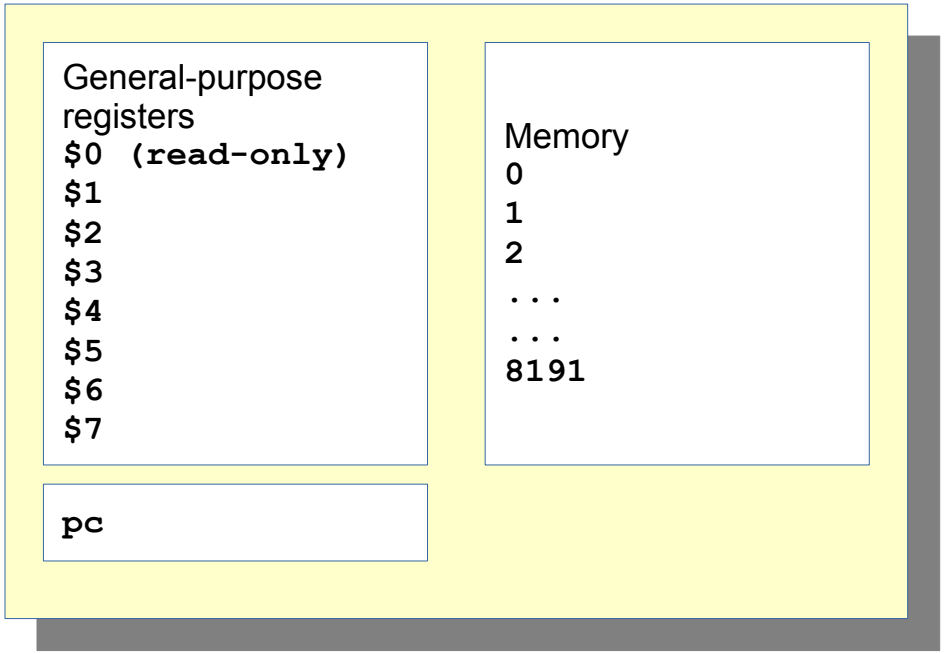
A *label* identifies a location in a program. Each label has a *name* and a *value*. The name is arbitrary. The value is the address of the memory where the label is defined.

Here, label **beginning** has value 1, and label **done** has value 4.

A label can be used as a destination of a jump, which is more convenient than giving the destination numerically.

n do?
of each

What's in the E20?

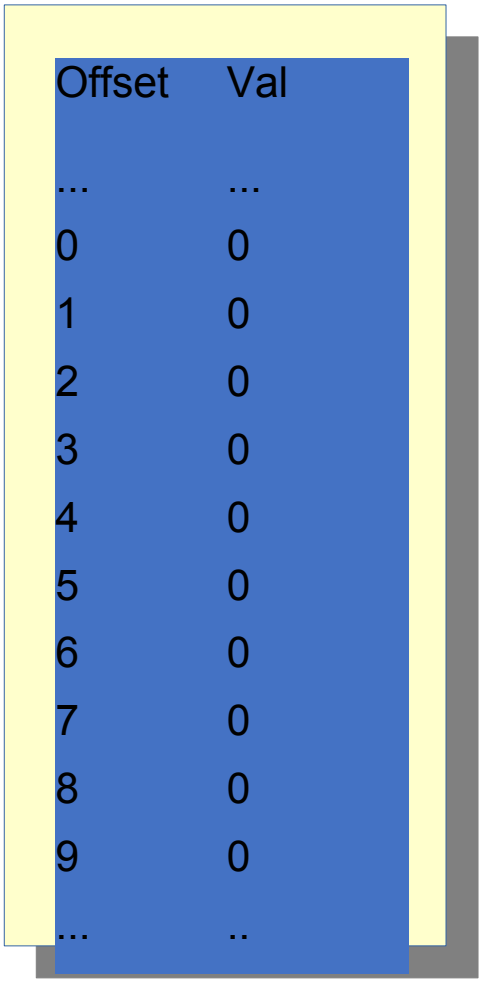


The E20 has eight 16-bit general purpose registers, of which $0 is read-only.
It has a 16-bit program counter.
It has 8192 16-bit memory cells.

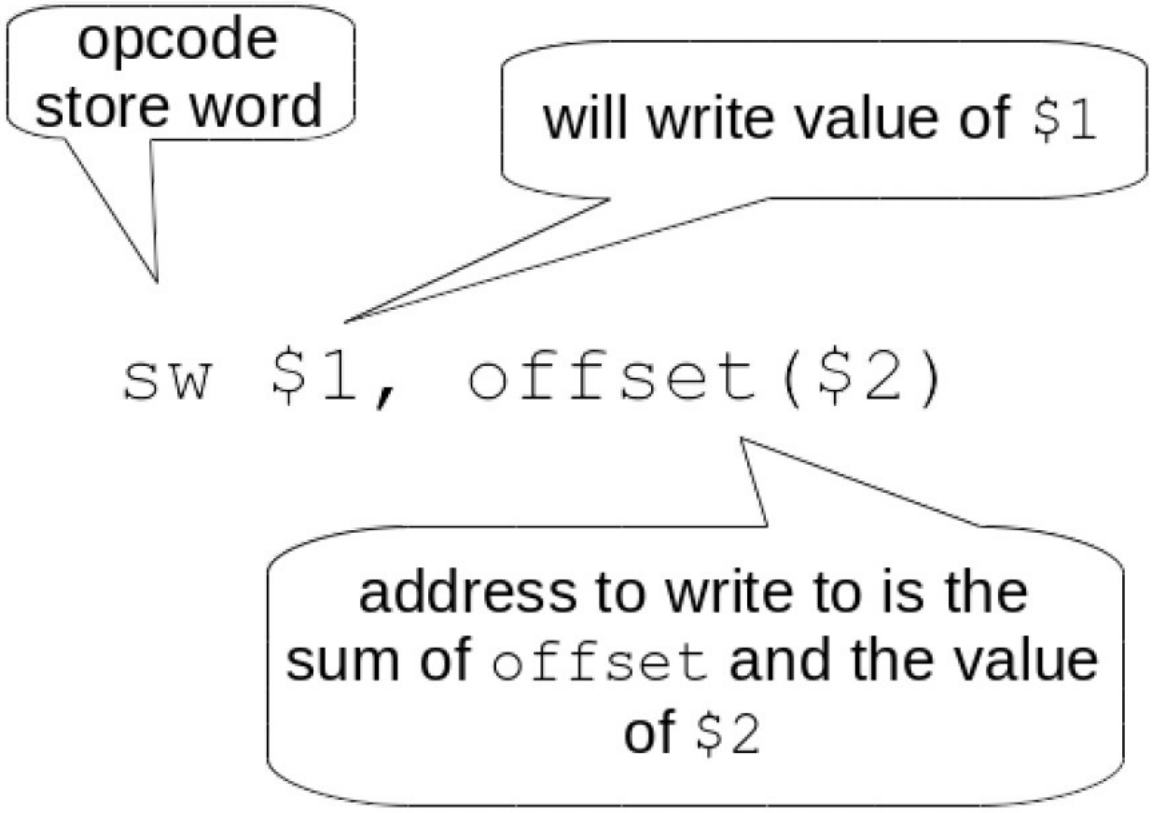
The E20 memory is an 8192-element array.

Each cell is 16-bits.

Each cell's value is initially zero.



Store Value from Register to Memory Location



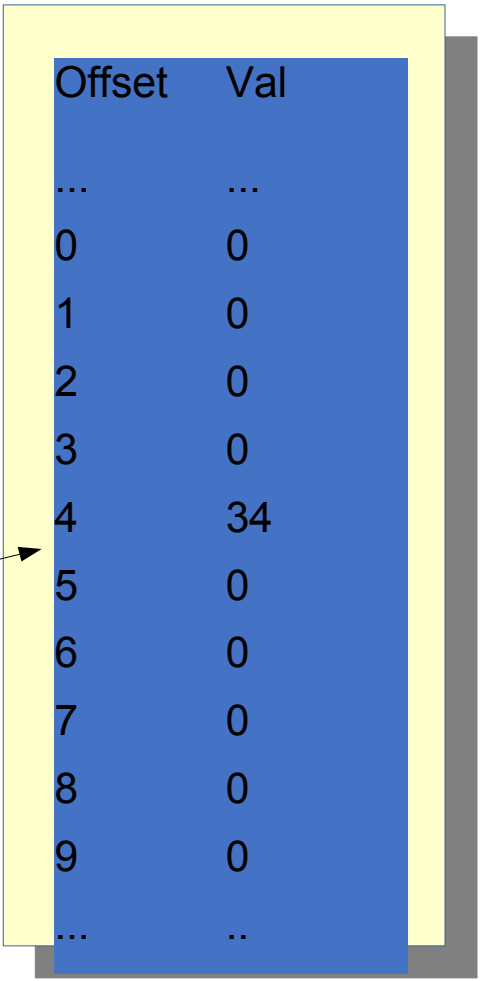
The E20 memory is an 8192-element array.

Each cell is 16-bits.

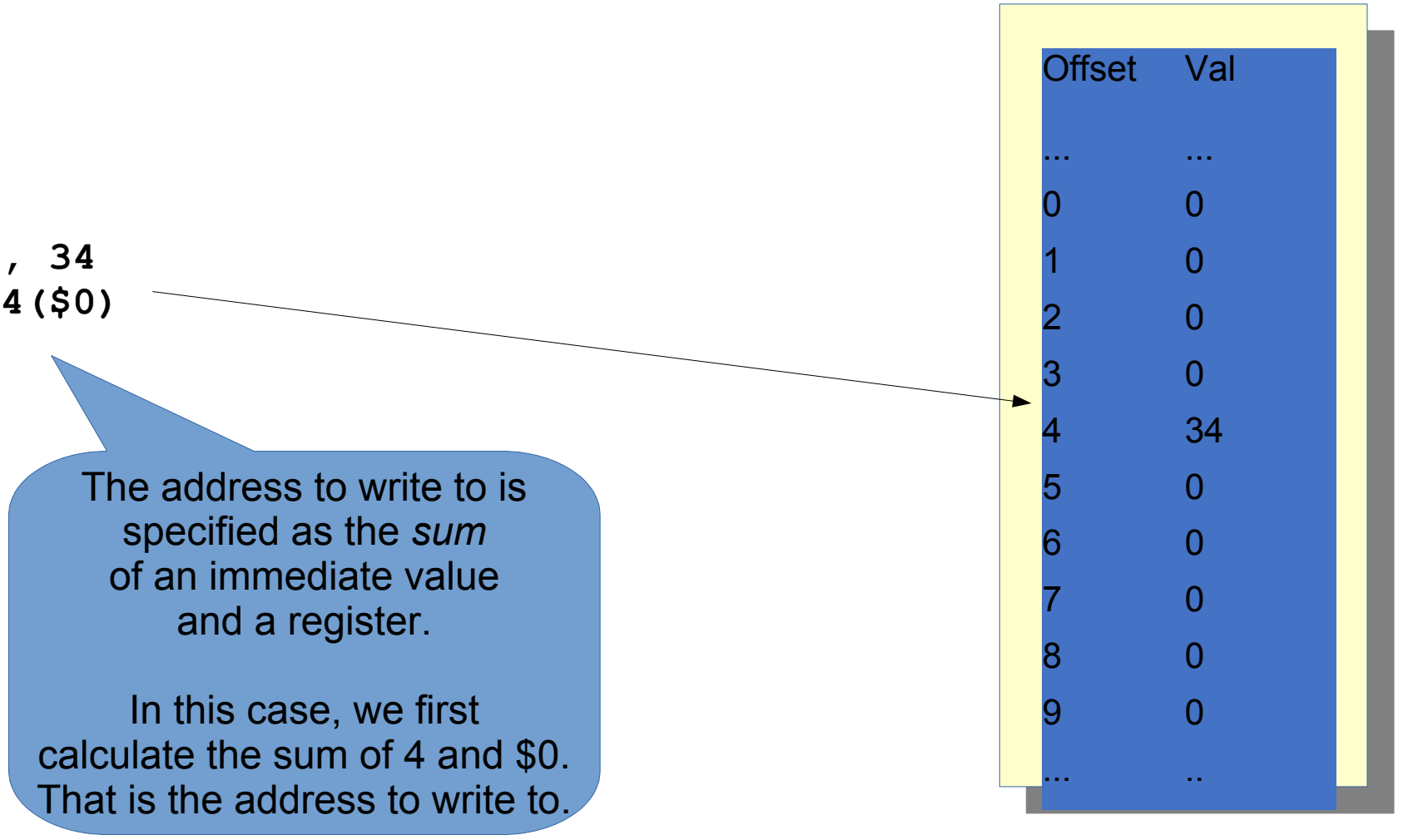
Each cell's value is initially zero.

Memory cells are changed with the **`sw`** instruction.

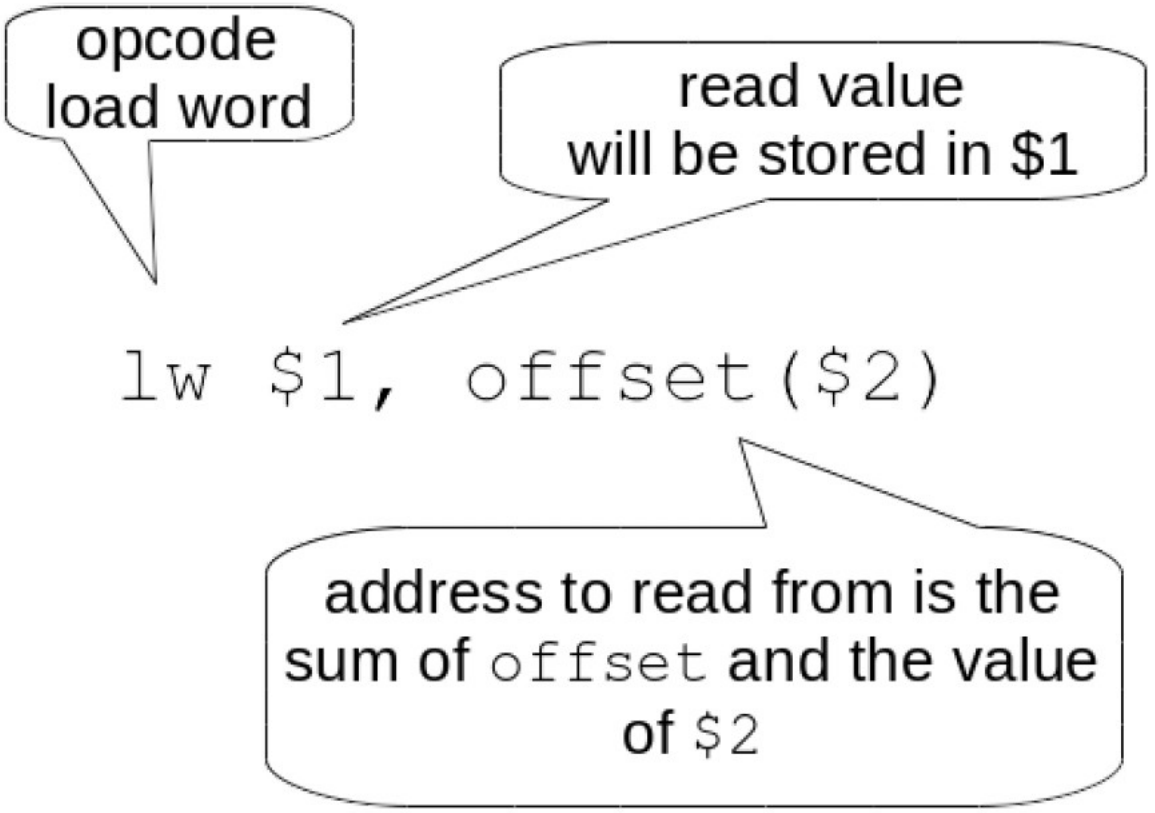
```
movi $1, 34
sw $1, 4($0)
```



```
movi $1, 34
sw $1, 4($0)
```



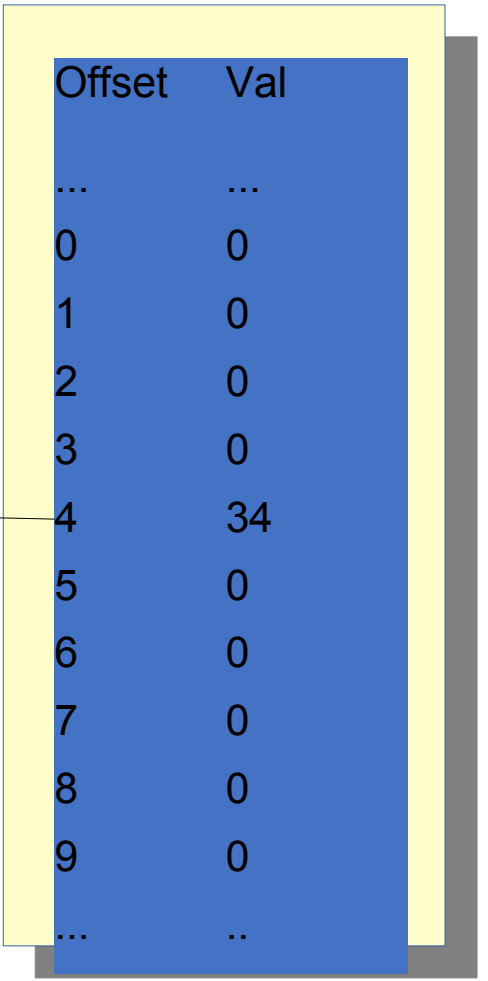
Load Value from Memory Location to Register



Memory cells are read with the **`lw`** instruction.
Given an address, they will copy the value of the memory cell into

```
lw $2, 4($0)
```

After this, $2 will have value 34.

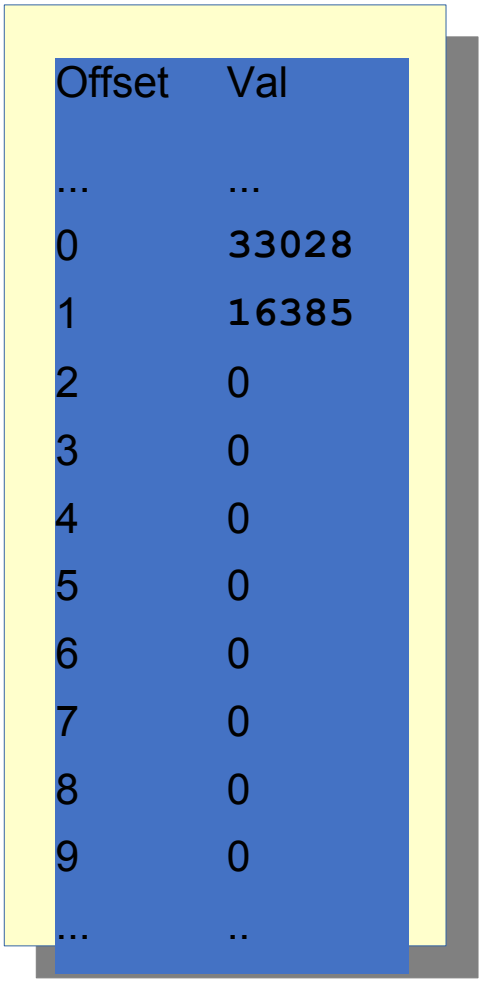


The address to read is specified as the *sum* of an immediate value and a register.

Actually, I lied: the initial value of all memory cells is *not* zero.
In reality, the program is loaded into memory, starting at address zero.

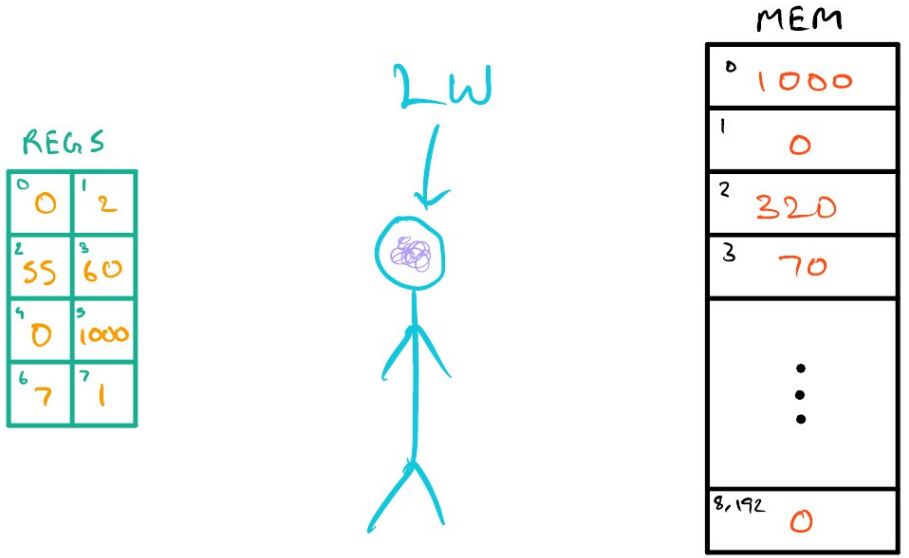
Consider this program:

Assembly code	Machine code (bin)	Machine code (dec)
`lw $2, 4($0)`	`1000000100000100`	`33028`
`halt`	`0100000000000001`	`16385`



ILLUSTRATING LW

lw $3, 3($0)



lw $3, 3($0)

Load value from R[$0]+3

= 0 + 3

= 3

REGS

0: 0	1: 2
2: 55	3: 60
4: 0	5: 1000
6: 7	7: 1

MEM

0	1000
1	0
2	320
3	70
⋮	
8,192	0

lw $3, 3($0)

REGS

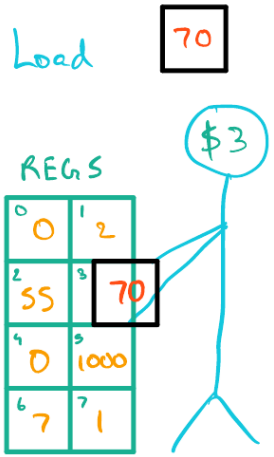
0: 0	1: 2
2: 55	3: 60
4: 0	5: 1000
6: 7	7: 1

MEM

0	1000
1	0
2	320
3	70
⋮	
8,192	0

lw $3, 3($0)

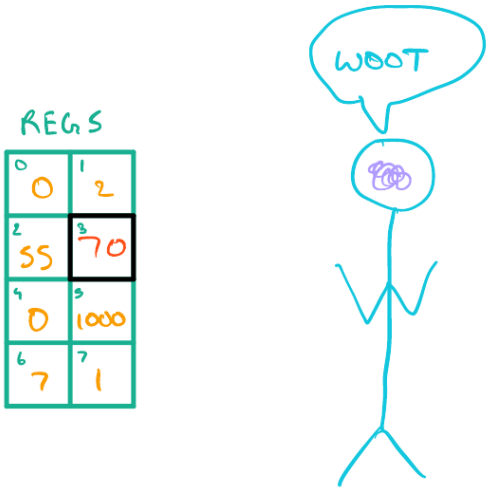
Load 70 in $3



MEM

0	1000
1	0
2	320
3	70
	⋮
8,192	0

lw $3, 3($0)

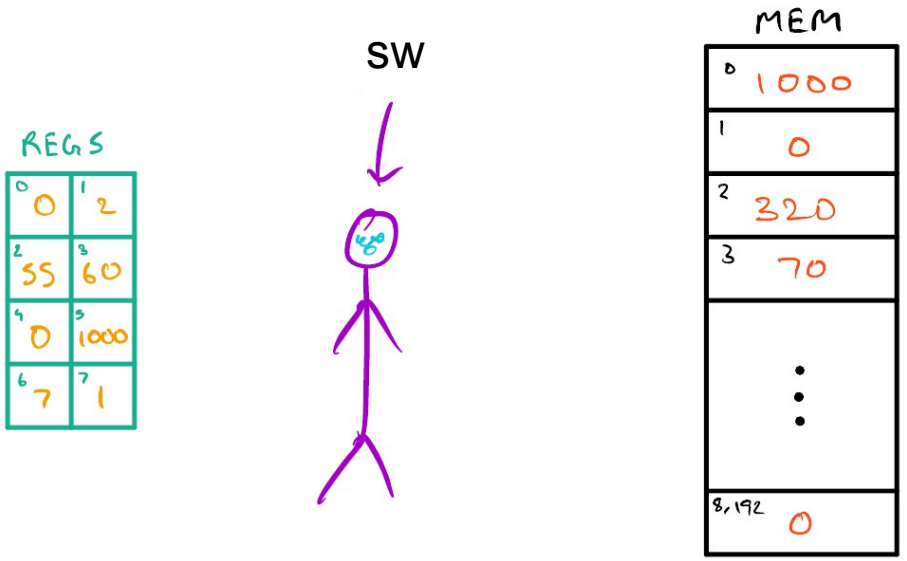


MEM

0	1000
1	0
2	320
3	70
	⋮
8,192	0

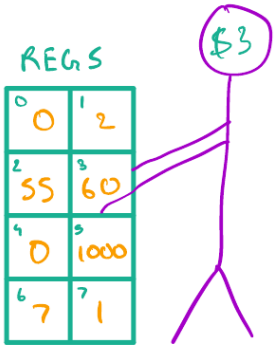
ILLUSTRATING SW

```
sw $3, 3($0)
```



sw $3, 3($0)

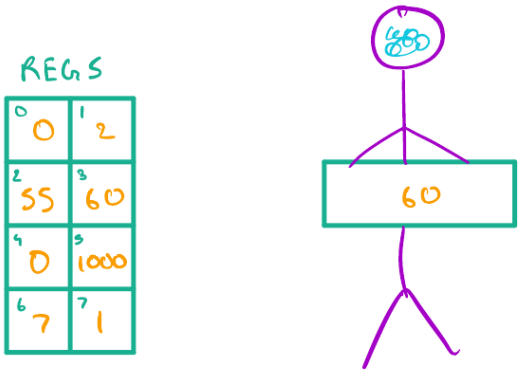
Store value from $3



MEM

0	1000
1	0
2	320
3	70
⋮	⋮
8,192	0

sw $3, 3($0)



MEM

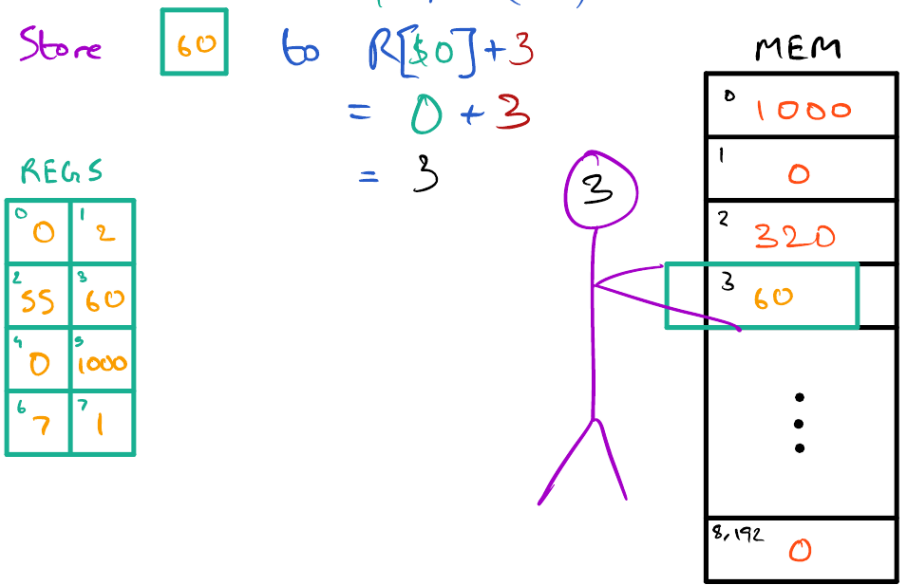
0	1000
1	0
2	320
3	70
⋮	⋮
8,192	0

sw $3, 3($0)

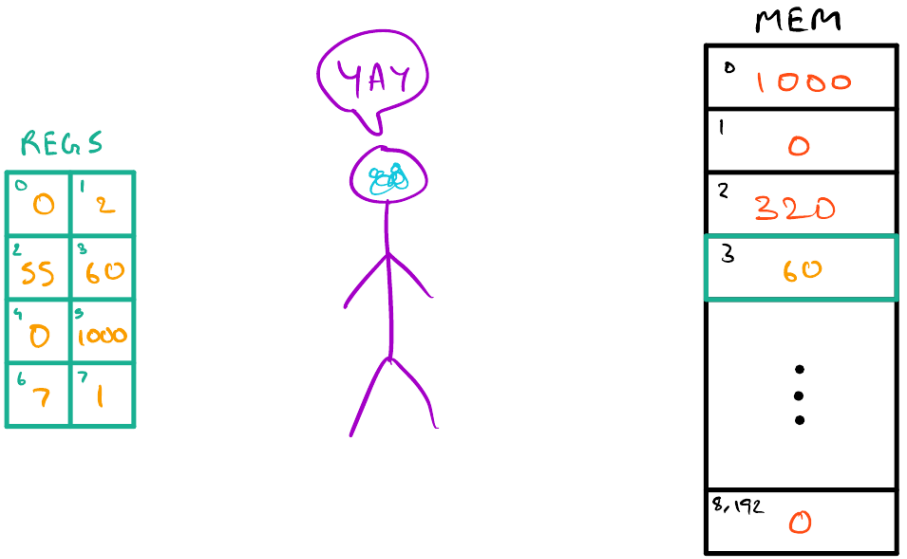
Store 60 to R[$0]+3

= 0 + 3

= 3



sw $3, 3($0)



Here's a sample of E20 assembly language.

Notice in particular the use of **lw** and **sw** to access memory.

```
    lw $1,var1($0)     # read from address var1 + 0
    lw $2,var2($0)     # read from address var2 + 0
    and $3, $1, $2     # AND the values together
    or $4, $1, $2      # then OR them together
    sw $3,var3($0)     # write the AND result into memory
    movi $5, var3      # put the address (not the value!) in $5
    addi $6, $5, var4

    halt               # program ends

var1:
    .fill 30

var2:
    .fill 5

var3:
    .fill 0

var4:
```

A label can be used anywhere an immediate is accepted, including with opcodes **sw**, **lw**, and **movi**.

Here, we read from the memory location identified by label **var1** using **lw**.

What does this program do?
What is the final value of each register?


```
addi $1, $1, -5
```

?


```
addi $1, $1, -5
```

?

Crash?
Random number?
-2?
0?
Other?

7 general-purpose read/write registers

$1 = 3

$2 = 0

$3 = 0

$4 = 0

$5 = 0

$6 = 0

$7 = 0

```
addi $1, $1, -5
```

?

Decimal	Binary (2's complement)
3	0000000000000011
-5	1111111111111011

7 general-purpose read/write registers

$1 = 3

$2 = 0

$3 = 0

$4 = 0

$5 = 0

$6 = 0

$7 = 0

```
addi $1, $1, -5
```

?

Decimal	Binary (2's complement)
3	0000000000000011
-5	1111111111111011
-2	1111111111111110

addi $1, $1, -5

This is the correct result, based on the binary arithmetic that we learned earlier.
But how should your simulator display it?

?

	Binary (2's complement)
3	0000000000000011
-5	1111111111111011
-2	1111111111111110

7 general-purpose read/write registers

- $1 = 3
- $2 = 0
- $3 = 0
- $4 = 0
- $5 = 0
- $6 = 0
- $7 = 0

```
addi $1, $1, -5
```

?

Decimal (unsigned)	Decimal (signed)	Binary (2's complement)
3	3	0000000000000011
65531	-5	1111111111111011
65534	-2	1111111111111110

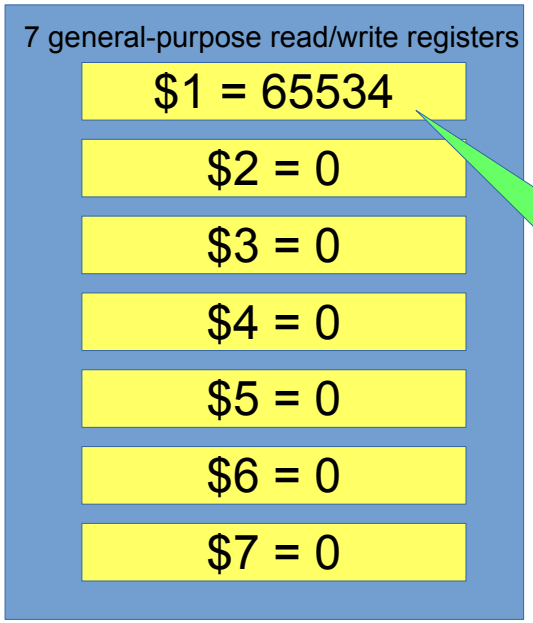
7 general-purpose read/write registers

- $1 = 3
- $2 = 0
- $3 = 0
- $4 = 0
- $5 = 0
- $6 = 0
- $7 = 0

```
addi $1, $1, -5
```

The E20 registers store 16-bit values. Your simulator should always display those values are unsigned decimal integers.
So this is what we should see!

Decimal (unsigned)	Deci	mplement)
3		0000000000000011
65531	-5	1111111111111011
65534	-2	1111111111111110



addi $1, $1, -5

This is the "wraparound" effect. Binary arithmetic means that small numbers "wrap" to high numbers and vice versa.

Deci	ment)
3	0000000000000011
-5	1111111111111011
-2	1111111111111110

Wraparound happen in any language that uses fixed-width integers.
C and C++ do this. Run this program and see for yourself.

```cpp
#include <iostream>
#include <cstdint>
using namespace std;
int main() {

    // uint16_t is a 16-bit unsigned int type
    uint16_t x = 3;
    x = x - 5;
    cout << "3-5=" << x << endl; // prints 65534

    uint16_t y = 60000;
    y = y + 7000;
    cout << "60000+7000=" << y << endl; // prints 1464


    return 0;
}
```

E20 Machine Code Translation

- E20 Manual
  - Instruction set (Starting from Page # 11)
  -
- Need to be careful while translating **jeq** instruction.

Summary of Today's Lecture

- We discussed HW3 problems which is due today.
- We introduced E20 Processor.
  - Registers – 1 immutable and 7 mutable 16-bit registers
  - Memory – 8192 memory cells, each cell is 16-bit,
  - Program Counter – 16-bit register, stores the memory address of the currently-executing instruction ignoring Most Significant 3 bits.
- We compared between E15 and E20.
- We introduced E20 Assembly language Instructions.
- We introduced E20 Machine Code Translation.
- HW4 will be assigned today.

Reference

- Please read the E20Manual.
  - Available on NYU Brightspace.
  -
- Please read the lecture slide (05-e20 V2).
  - Available on NYU Brightspace

Plan for next lecture

- Branching and Iterative Statements
  - Comparison
  - Jump
- Array.
  - Using .fill and labels
- Subroutines/Functions
  - Using jal and jr instructions
- Recursive functions using stacks, jal, jr.
- Hardware Implementation: E20 Single Cycle.

E20 Recursion

What is a function?

- A function needs to be know where it was called from, and go back there when it's done
  - use **`jal`** and **`jr`**
  - return address stored in $7
- A function needs to accept parameters and return a result
  - we can decide to use registers
  - parameters in $1, $2, $3, ...
  - return value in $1

```
    movi $1, 1
    jal quadruple
    add $2, $0, $1    # save result

    movi $1, 5
    jal quadruple

    add $4, $2, $1     # $4 = 4*1 + 4*5
    halt

quadruple:
    add $1, $1, $1
    add $1, $1, $1
    jr $7
```

What is a function?

- But what happens when a function calls a function?
  - Saved return address and parameters will be lost

What is a function?

- But what happens when a function calls a function?
  - Saved return address and parameters will be lost

What is a function?

- But what happens when a function calls a function?
  - Saved return address and parameters will be lost

What is a function?

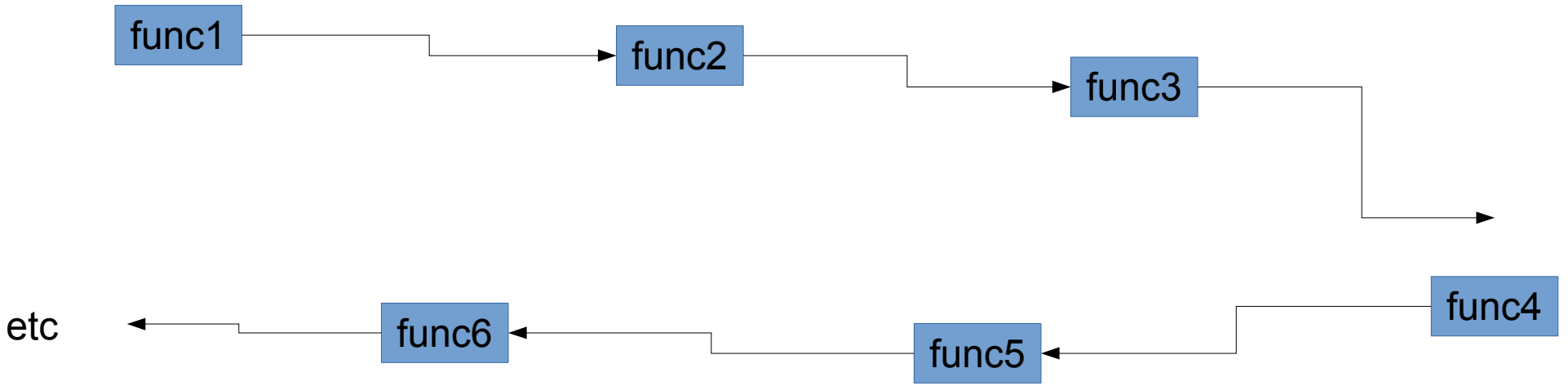
- But what happens when a function calls a function?
  - Saved return address and parameters will be lost

What is a function?

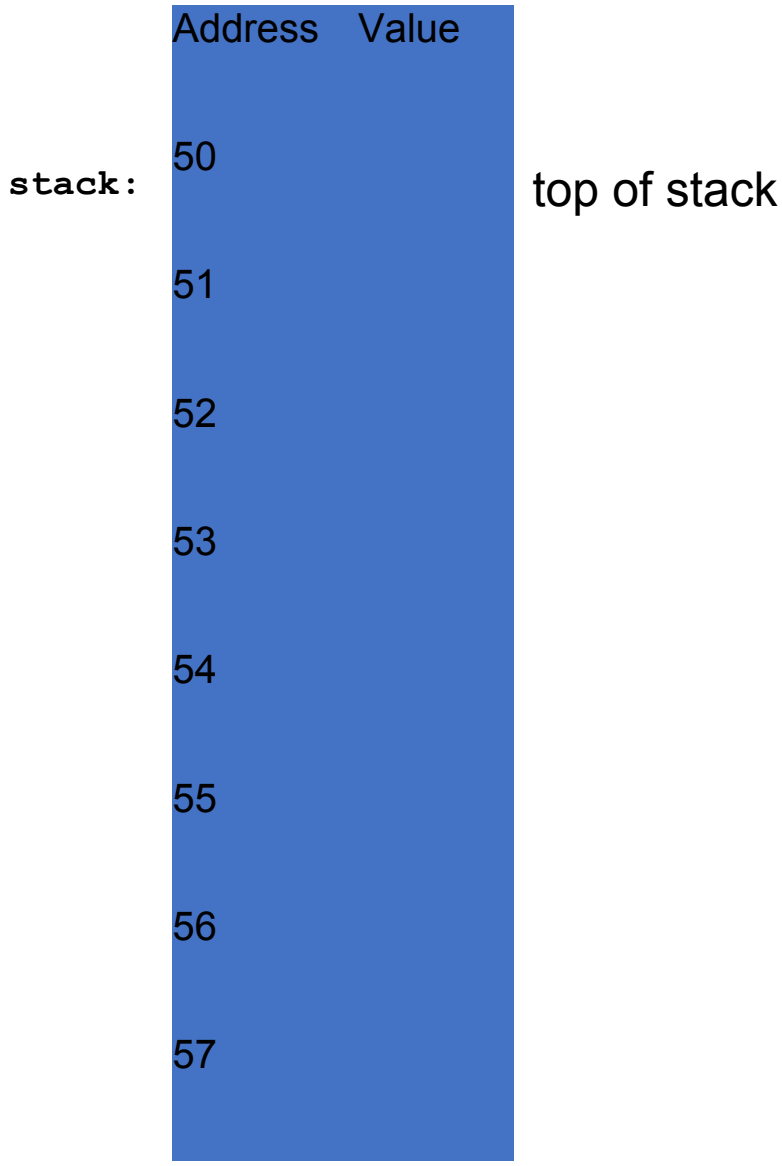
- But what happens when a function calls a function?
  - Saved return address and parameters will be lost

What is a function?

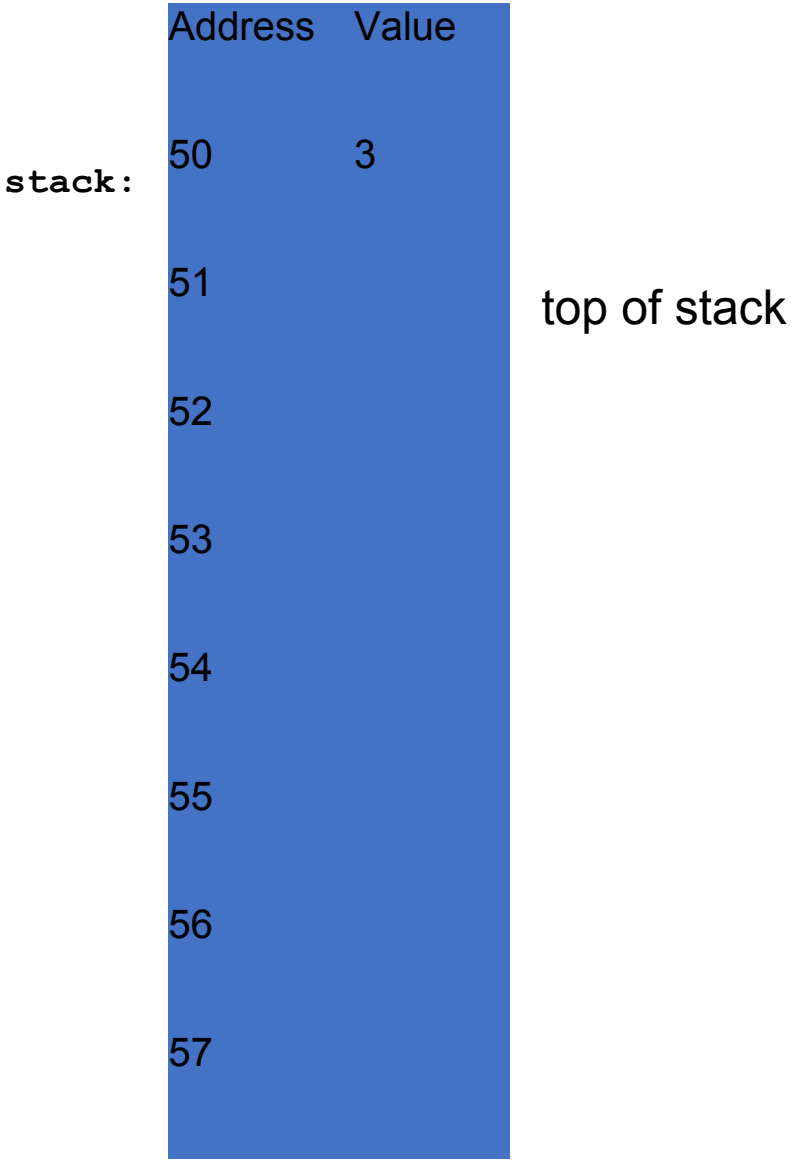
- But what happens when a function calls a function?
  - Saved return address and parameters will be lost
  - The chain of function can be arbitrarily long
- Especially if recursion!



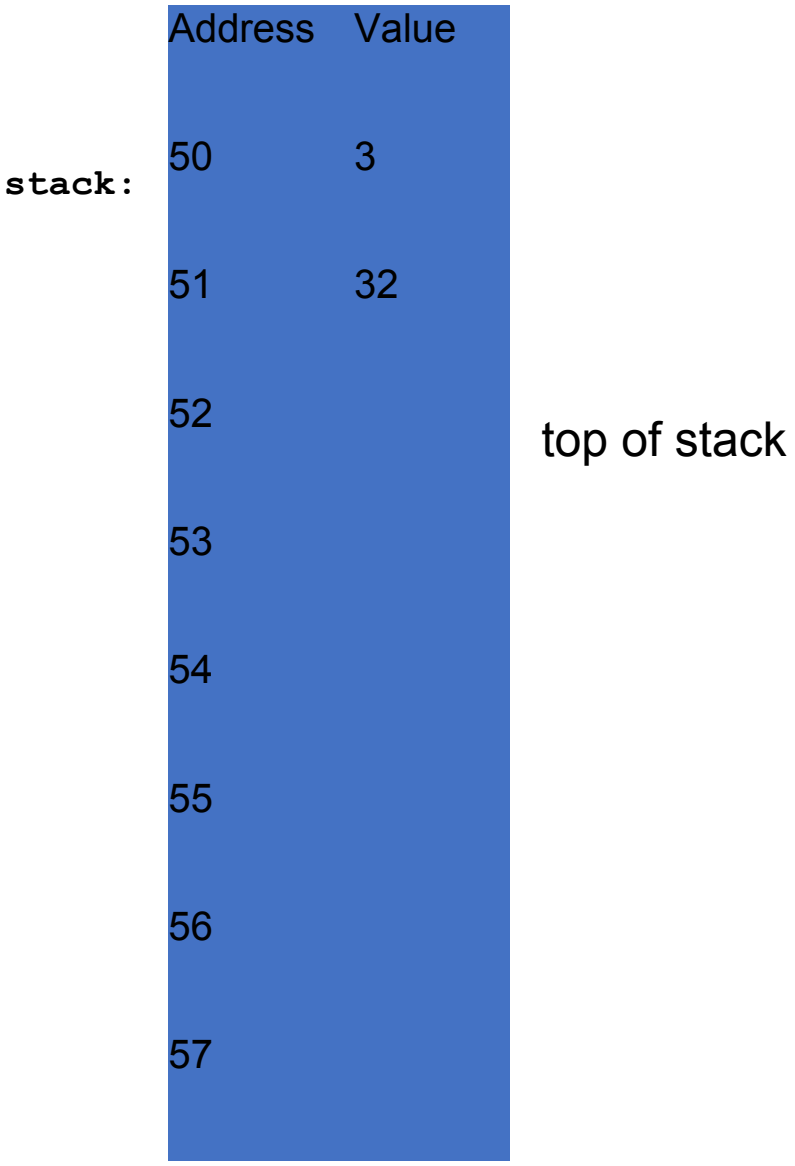
fib(3)



fib(3)
save argument on stack



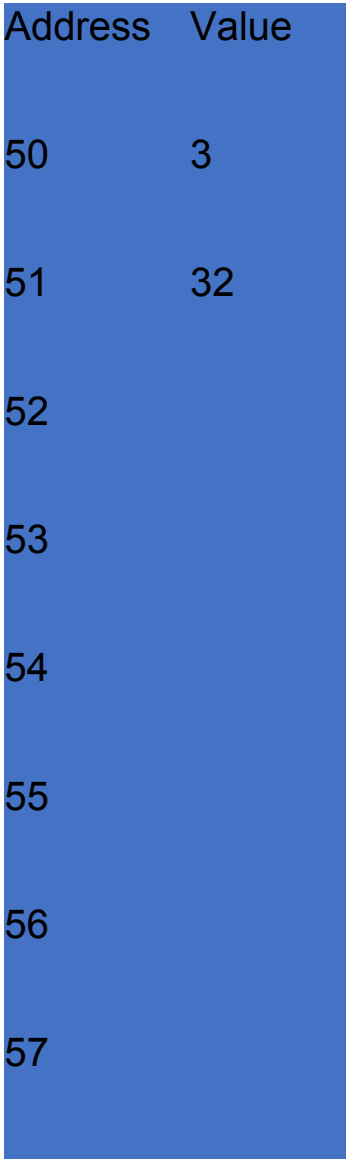
fib(3)
save argument on stack
save return address on stack



fib(3)
save argument on stack
save return address on

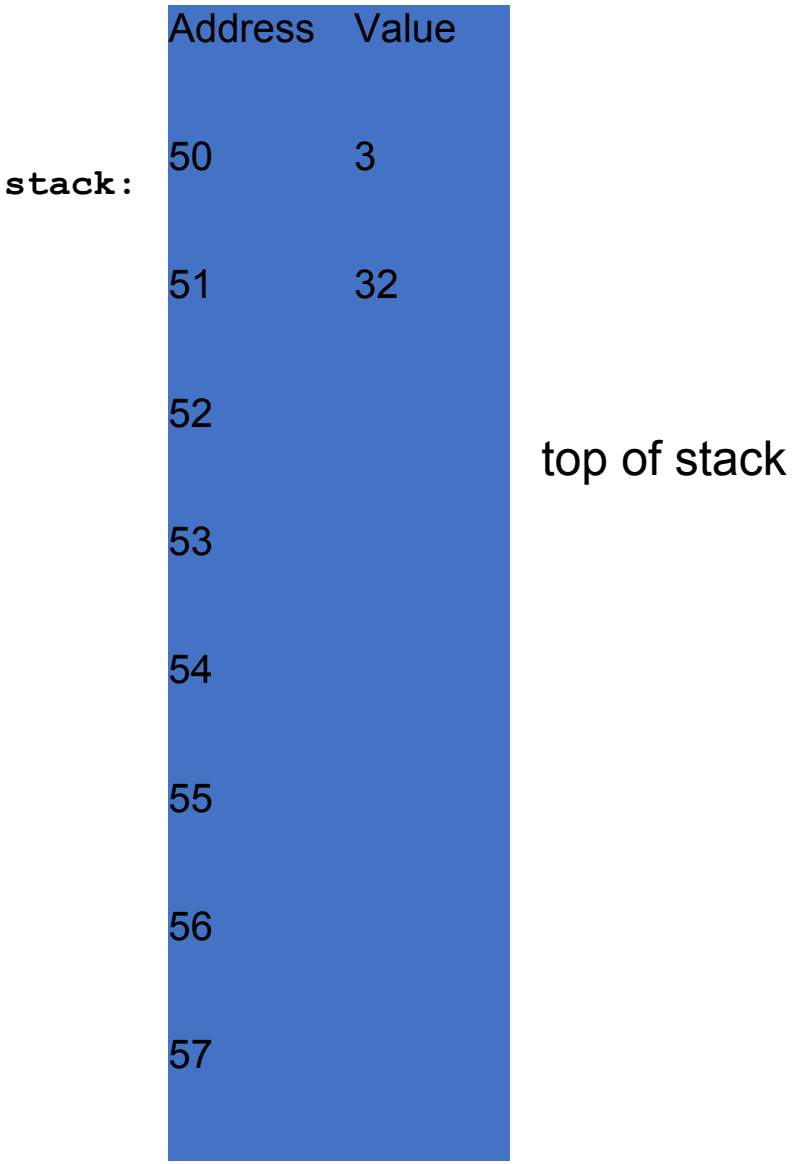
Note: return address is at topofstack-1; and argument is at topofstack-2

tack:

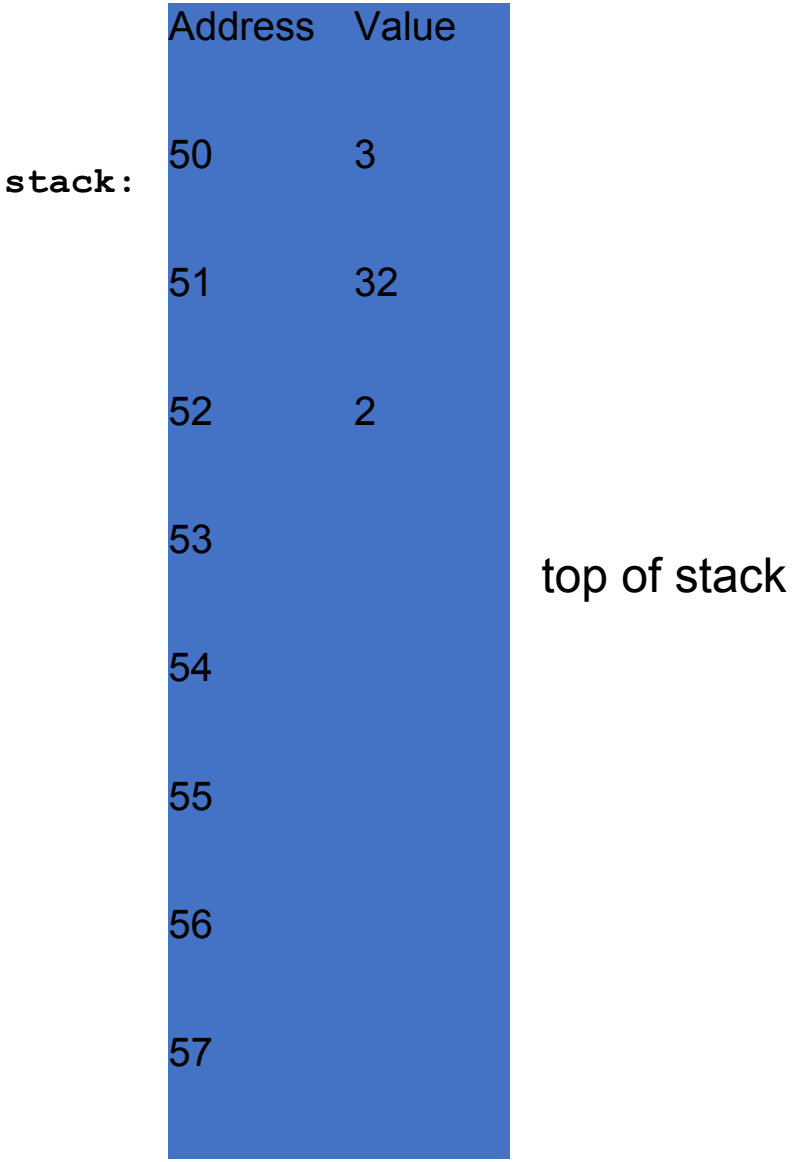


top of stack

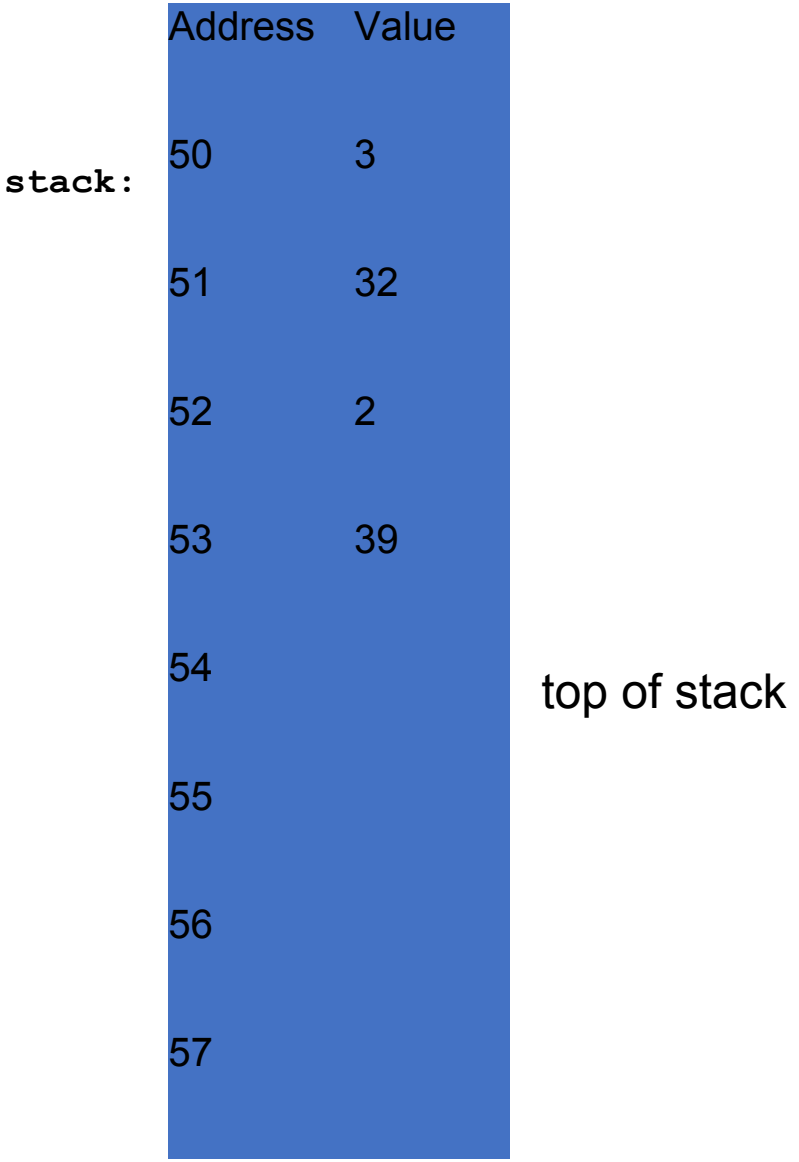
fib(3)
save argument on stack
save return address on stack
call fib(2)



fib(3)
save argument on stack
save return address on stack
call fib(2)
save argument on stack



fib(3)
    save argument on stack
    save return address on stack
    call fib(2)
        save argument on stack
        save return address on stack



```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
```

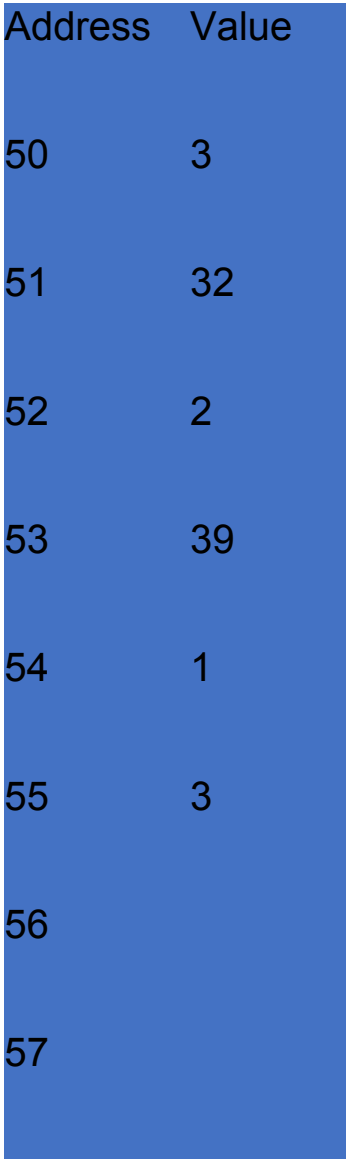
`stack:`

Address	Value	
50	3	
51	32	
52	2	
53	39	
54	1	
55	3	
56		top of stack
57		

fib(3)
save argument on stack
save return address on stack
call fib(2)
save argument on s
save return addres
call fib(1)
save argument on ck
save return add ss on stack
base case!

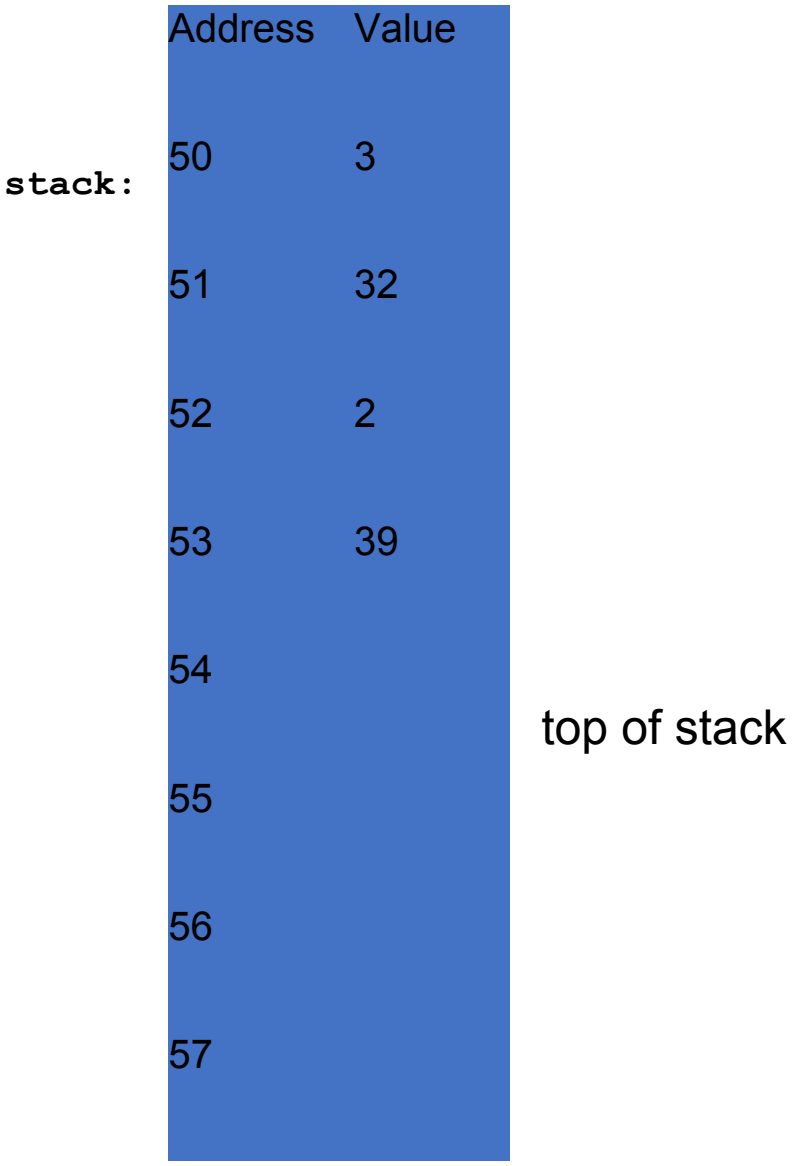
It's a base case, so we don't recurse.

`stack:`



top of stack

```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
```



```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
                save result from fib(1) on stack
```

stack:

Address	Value
50	3
51	32
52	2
53	39
54	1
55	
56	
57	

top of stack

```
fib(3)
    save argument on stack
    save return address on stack
    call fib(2)
        save argument on stack
        save return address on stack
        call fib(1)
            save argument on stack
            save return address on stack
            base case!
            remove return address from stack
            remove argument from stack
            jump to saved return address
        save result from fib(1) on stack
        call fib(0)
            [same as above]
```

`stack:`

Address	Value
50	3
51	32
52	2
53	39
54	1
55	1
56	
57	

top of stack

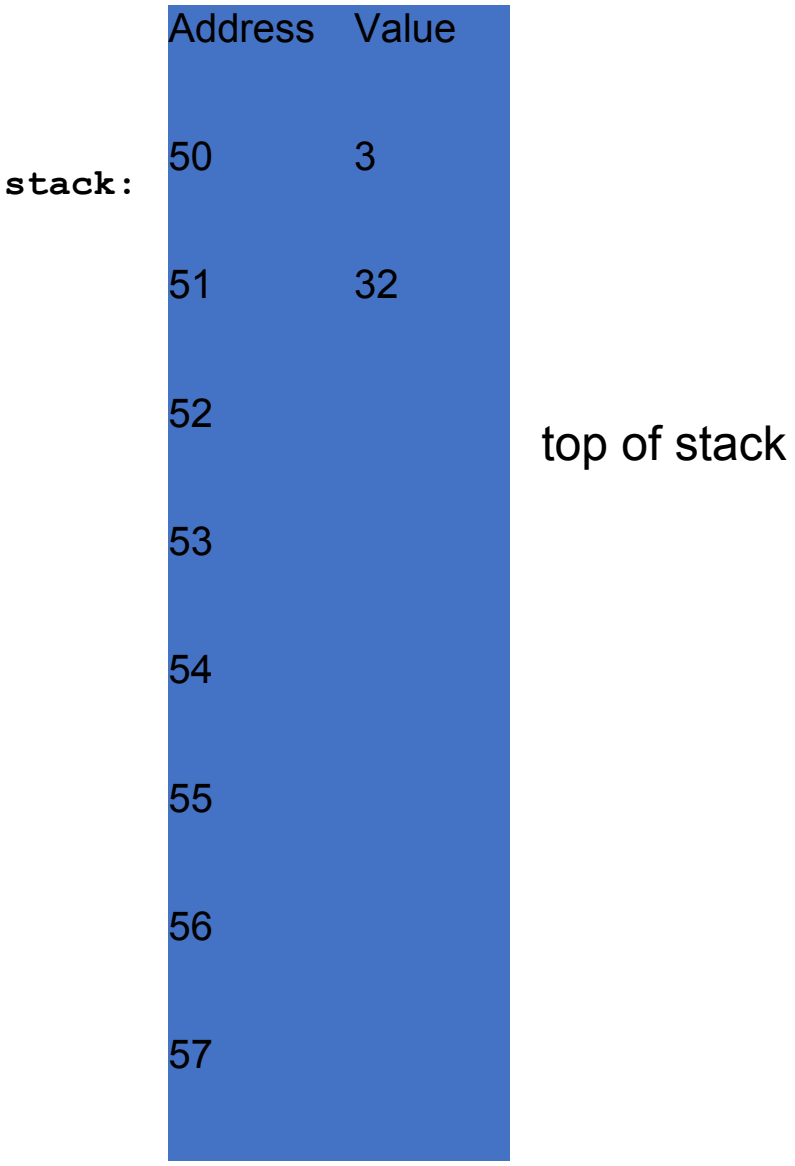
```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
                save result from fib(1) on stack
                call fib(0)
                        [same as above]
                remove saved results from stack, sum
```

`stack:`

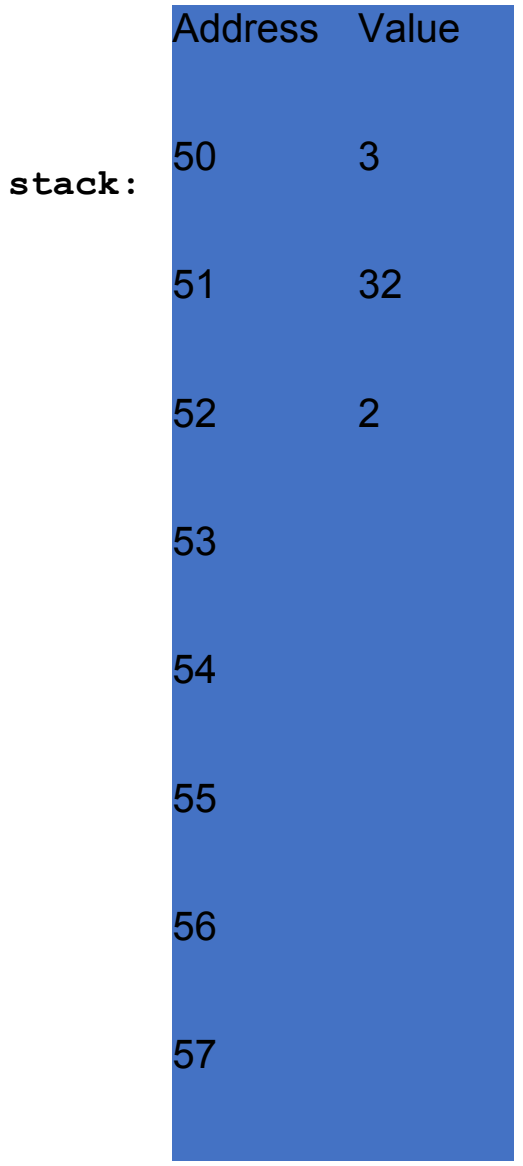
Address	Value
50	3
51	32
52	2
53	39
54	
55	
56	
57	

top of stack

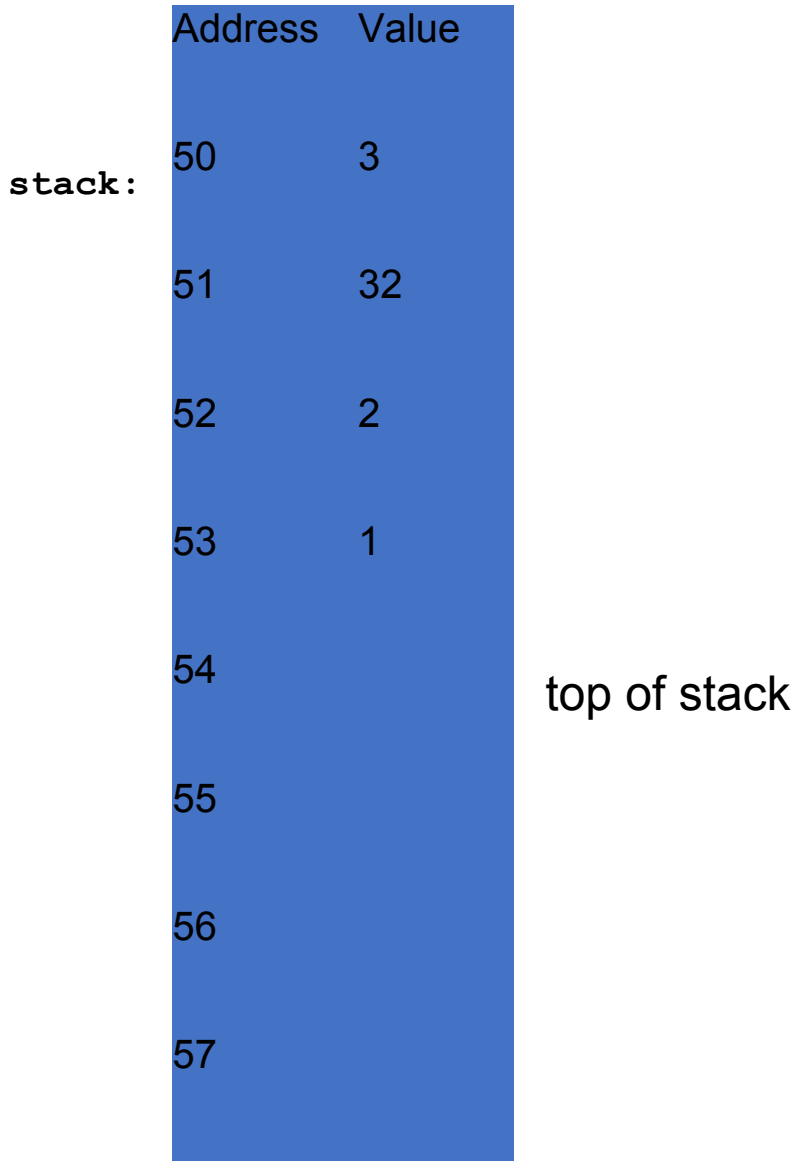
```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
                save result from fib(1) on stack
                call fib(0)
                        [same as above]
                remove saved results from stack, sum
                remove return address from stack
                remove argument from stack
                jump to saved return address
```



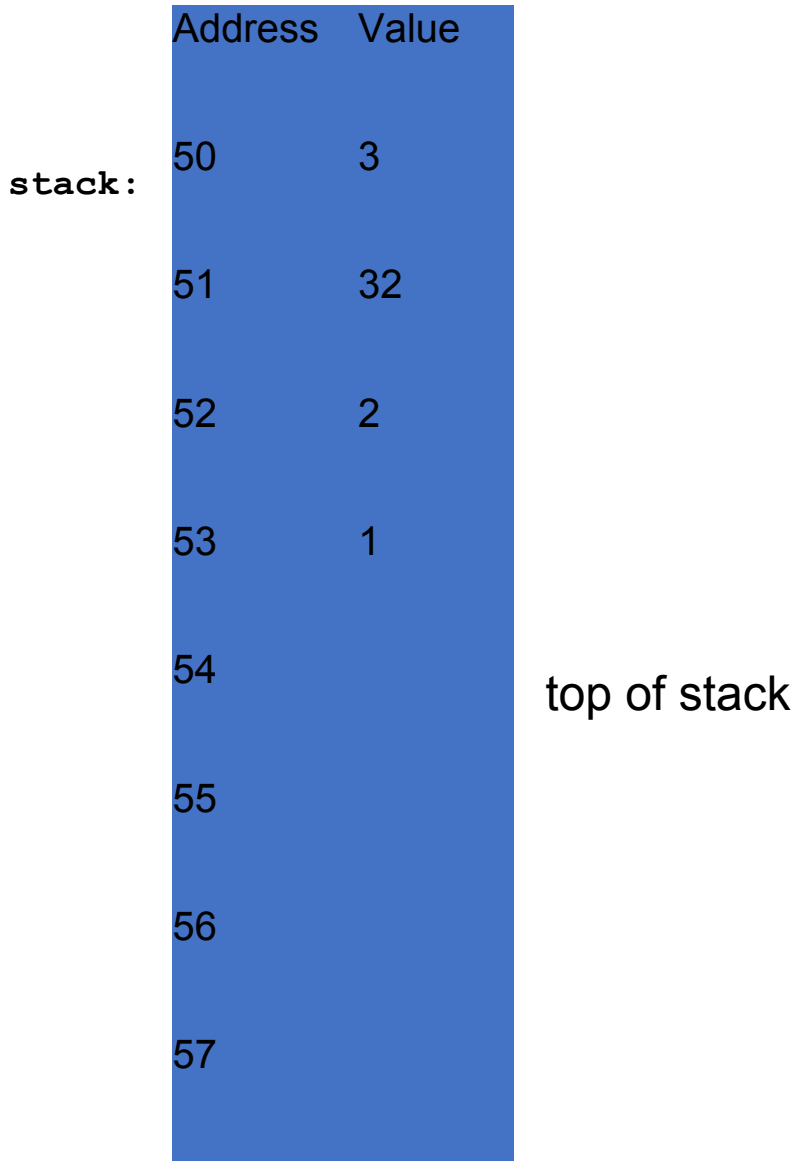
```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
                save result from fib(1) on stack
                call fib(0)
                        [same as above]
                remove saved results from stack, sum
                remove return address from stack
                remove argument from stack
                jump to saved return address
        save result from fib(2) on stack
```



```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
                save result from fib(1) on stack
                call fib(0)
                        [same as above]
                remove saved results from stack, sum
                remove return address from stack
                remove argument from stack
                jump to saved return address
        save result from fib(2) on stack
        call fib(1)
                [same as above]
```



```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
                save result from fib(1) on stack
                call fib(0)
                        [same as above]
                remove saved results from stack, sum
                remove return address from stack
                remove argument from stack
                jump to saved return address
        save result from fib(2) on stack
        call fib(1)
                [same as above]
        removed save results from stack, sum
```



```
fib(3)
        save argument on stack
        save return address on stack
        call fib(2)
                save argument on stack
                save return address on stack
                call fib(1)
                        save argument on stack
                        save return address on stack
                        base case!
                        remove return address from stack
                        remove argument from stack
                        jump to saved return address
                save result from fib(1) on stack
                call fib(0)
                        [same as above]
                remove saved results from stack, sum
                remove return address from stack
                remove argument from stack
                jump to saved return address
        save result from fib(2) on stack
        call fib(1)
                [same as above]
        removed save results from stack, sum
        remove return address from stack
        remove argument from stack
        jump to saved return address
```